

Weekly Internship Report

General Information

Name: FROEHLI Jean-Baptiste

Period: Week 3 - 21/04/2025 to 25/04/2025

Weekly Objectives

- **Objective 1:** Focus on attacks and understand how they work
- **Objective 2:** Continue to implement labs on my image datasets
- **Objective 3:** Benchmark the various attacks

Summary

Dataset

Special point: The dataset I was using has been removed from kaggle. So I found another one, with more images, but which were not prepared like the previous one.

Source : ADNI database - [ADNI Website](#)

New kaggle source : [Dataset link](#) - Author : [Abdullah Tauseef](#)

Infos : ~490mo, +34.0k images

This dataset provides a collection of preprocessed MRI brain scan images from the ADNI (Alzheimer's Disease Neuroimaging Initiative) project

Dataset structure

The images are arranged and classified in different categories in four directories. So I made a script to rearrange them like the previous dataset:

Pictures are referenced in a `train.csv` file and are renamed according to their diagnosis (ex: AD-0001.jpg, LMCI-2890 stand for `diagnosis_label-number_of_image`)

train.csv	
id_code (string)	diagnosis (int)
AD-3471	4
CN-1819	0
LMCI-0760	3
...	...

Here are the different classifications and their diagnosis id

- CN - Cognitively Normal : `diagnosis=0` ; 6464 images
- EMCI - Early Mild Cognitive Impairment : `diagnosis=1` ; 9600 images
- LMCI - Late Mild Cognitive Impairment : `diagnosis=2` ; 8960 images
- AD - Alzheimer's Disease : `diagnosis=3` ; 8960 images

Attack studied

I mainly focused on **Adversarial Attacks** and more specifically on **evasion attacks**.

General Definition : An **adversarial attack** is an attempt to manipulate a machine learning model via specifically calculated perturbations in the input data.

Evasion attacks : Evasion attacks target the model **at the time of inference**, without affecting the training. The aim is to slightly modify the input (x) into a version (x') so that the model is wrong, while keeping the modifications imperceptible.

How evasion attacks work

Principle

We seek to construct an input (x') such that :

$$f(x') \neq f(x) \text{ et } \|x - x'\| < \epsilon$$

where (ϵ) is a tolerated disturbance threshold, and (f) is the model.

Attack scenarios

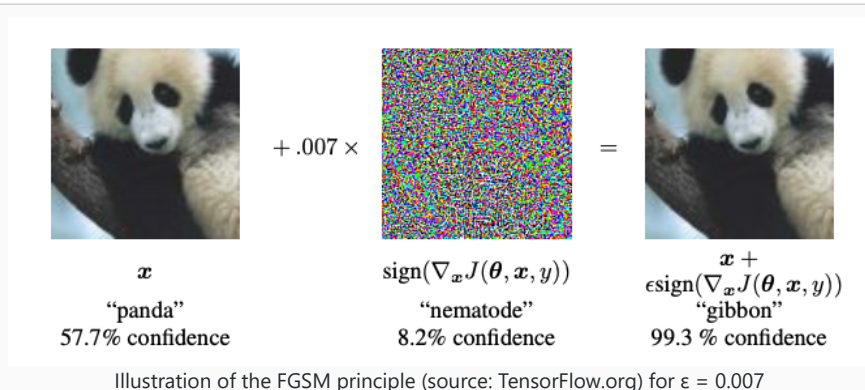
- **White box** : the attacker knows the model (f) , the weights (θ) , and the loss function (J) .
- **Black box** : the attacker only has access to the model's outputs.
- **Grey box** : the attacker has partial knowledge (for example, access to gradients but not to exact weights)

Evasion attack types

In general, there are 4 main types of evasion attack

Fast Gradient Sign Method (FGSM)

- **Principle**: Attack based on gradient, simple but efficient
- **How it works**:
 - Calculates the gradient of the loss function with respect to the input image
 - Add a small perturbation in the direction that maximises the error
 - Formula: $x_{adv} = x + \epsilon * \text{sign}(\nabla_x J(\theta, x, y))$
 - $\epsilon = 0.2$ for my tests
- **Characteristics**:
 - One-shot attack
 - Disturbances often visible to the naked eye
 - Quick to calculate



Projected Gradient Descent (PGD)

- **Principle**: More powerful iterative version of FGSM
- **How it works**:
 - Apply FGSM in several small steps (10 iterations in my tests)
 - Formula: $x_{adv}(t+1) = \text{Proj}(x_{adv}(t) + \alpha * \text{sign}(\nabla_x J(\theta, x_{adv}(t), y)))$
 - After each step, projection into a field ϵ
- **Characteristics**:
 - More sophisticated than FGSM
 - More discrete disturbances
 - More expensive to calculate

Carlini & Wagner (C&W)

- **Principle**: Optimization-based attack designed to bypass many existing defenses
- **How it works**:
 - Defines the attack as an optimization problem: find the smallest perturbation (δ) such that the perturbed input is misclassified
 - Minimizes a custom loss function that balances the size of the perturbation and the likelihood of misclassification
 - Often uses L2 norm but can also be adapted to L0 or L ∞

- Formula (L2 norm): $\min \|x - x'\|_2 + c \cdot f(x')$

- **Characteristics:**
 - Highly effective and stealthy (very small perturbations)
 - Bypasses defensive distillation and other robust models
 - Computation-heavy due to optimization process

DeepFool

- **Principle:** Iterative attack that assumes the model is approximately linear around the input
- **How it works:**
 - Computes the minimal perturbation required to cross the decision boundary
 - At each step, approximates the classifier by a linear model and moves toward the closest boundary
 - Stops when the label changes
- **Characteristics:**
 - Produces minimal and often imperceptible perturbations
 - Works well for untargeted attacks
 - Relatively fast and effective

Black Box attacks

- **Principle:** Attacks where the attacker has no access to model internals (weights, gradients, etc.)
- **How it works:**
 - Uses query-based approaches: observe outputs for various inputs to infer model behavior
 - Can also train a surrogate model using the outputs of the target model, and then attack the surrogate
 - Examples include Transfer Attacks, Zeroth-order Optimization (ZOO), and Boundary Attacks
- **Characteristics:**
 - More realistic in real-world scenarios
 - Usually require a large number of queries
 - Less efficient than white-box attacks but still dangerous

Common defenses

Adversarial Training, Defensive distillation, Gaussian Noise/Dropout, Gradient Smoothing, Adversarial Examples detection, randomization, etc...

Common Usages & Impacts

- **Facial Recognition:** modified glasses that can fool a system.
- **Autonomous vehicles:** disruption of road signs
- **Biometrics:** false fingerprints.

Examples

- Attack on Google Cloud Vision: a modified panda becomes a gibbon.
- Masks with special patterns to fool facial recognition

Results

Training and tests carried out locally on Nvidia RTX 4060 Laptop GPU, Python 3.10.0

20 epochs — Optimizer Adam - CNN : 2*conv+pooling, 2 fully conn., fn: ReLU/p>

Batch size : 32

Avg Epoch Time : 75s

Clean Results

```
[*] Clean evaluation:
Accuracy: 0.8124
Average inference time per batch: 0.0016 seconds
Classification Report:
precision  recall  f1-score  support
0         0.93     0.94     0.93     1307
```

1	0.82	0.80	0.81	1918
2	0.75	0.76	0.75	1847
3	0.78	0.79	0.78	1725
accuracy			0.81	6797
macro avg	0.82	0.82	0.82	6797
weighted avg	0.81	0.81	0.81	6797

[TIME] evaluate_model executed in 13.88 seconds

Attack Implementation

FGSM

[*] Attack: FGSM ($\epsilon=0.2$)
Accuracy: 0.1081
Average attack+inference time per batch: 0.0724 seconds

Classification Report:

	precision	recall	f1-score	support
0	0.22	0.08	0.12	1307
1	0.09	0.09	0.09	1918
2	0.08	0.12	0.09	1847
3	0.14	0.14	0.14	1725
accuracy			0.11	6797
macro avg	0.13	0.11	0.11	6797
weighted avg	0.13	0.11	0.11	6797

[TIME] test_evasion_attack executed in 25.90 seconds

PGD

[*] Attack: PGD ($\epsilon=0.2$, iter=10)
Accuracy: 0.0881
Average attack+inference time per batch: 0.3330 seconds

Classification Report:

	precision	recall	f1-score	support
0	0.09	0.05	0.06	1307
1	0.09	0.09	0.09	1918
2	0.06	0.09	0.07	1847
3	0.13	0.12	0.12	1725
accuracy			0.09	6797
macro avg	0.09	0.09	0.09	6797
weighted avg	0.09	0.09	0.09	6797

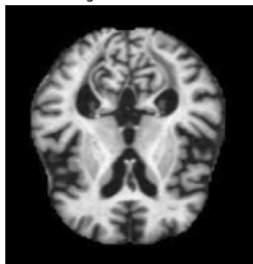
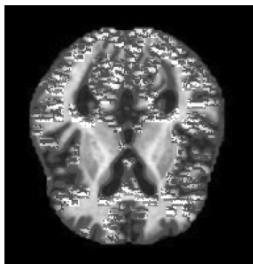
[TIME] test_evasion_attack executed in 81.36 seconds

Accuracy Metrics:
Initial clean accuracy: 0.8124
Accuracy under FGSM attack: 0.1081 (Drop: 0.7043)
Accuracy under PGD attack: 0.0881 (Drop: 0.7243)

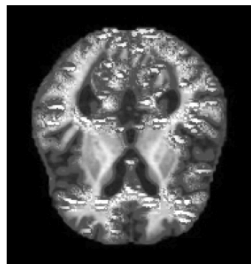
Performance Metrics:
Standard training time: 1526.77 seconds
Average clean inference time: 0.0016 seconds per batch
Average FGSM attack+inference time: 0.0724 seconds per batch
Average PGD attack+inference time: 0.3330 seconds per batch

Images

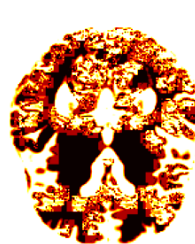
Original (Label: 3)

FGSM Perturbed $\epsilon=0.4$ 

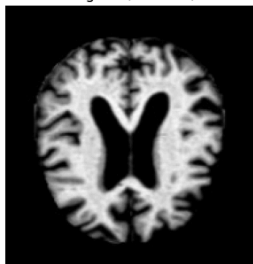
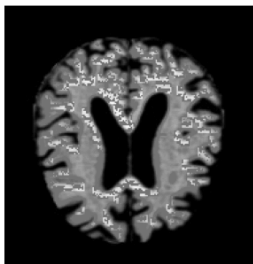
FGSM Difference

PGD Perturbed $\epsilon=0.4$ 

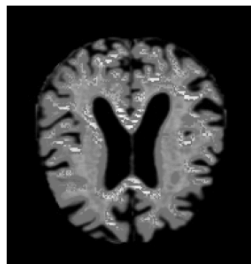
PGD Difference



Original (Label: 3)

FGSM Perturbed $\epsilon=0.4$ 

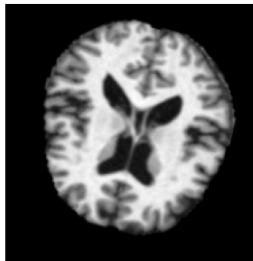
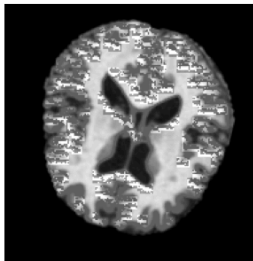
FGSM Difference

PGD Perturbed $\epsilon=0.4$ 

PGD Difference



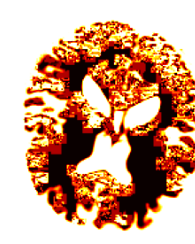
Original (Label: 2)

FGSM Perturbed $\epsilon=0.4$ 

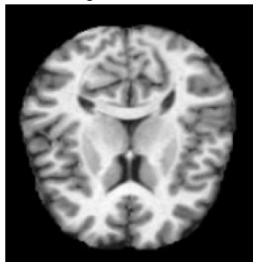
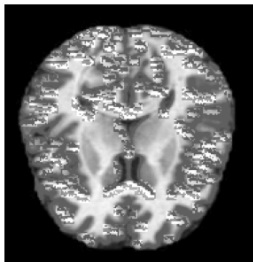
FGSM Difference

PGD Perturbed $\epsilon=0.4$ 

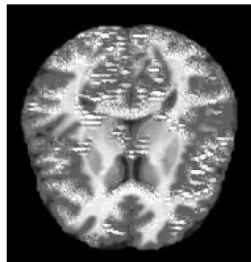
PGD Difference



Original (Label: 1)

FGSM Perturbed $\epsilon=0.4$ 

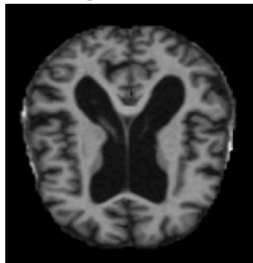
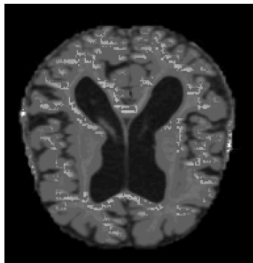
FGSM Difference

PGD Perturbed $\epsilon=0.4$ 

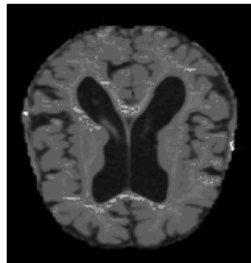
PGD Difference



Original (Label: 2)

FGSM Perturbed $\epsilon=0.4$ 

FGSM Difference

PGD Perturbed $\epsilon=0.4$ 

PGD Difference

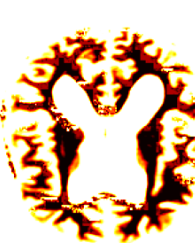
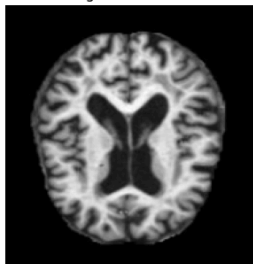
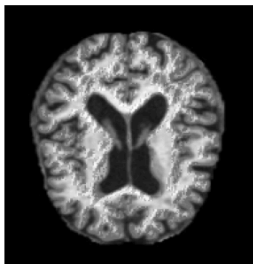
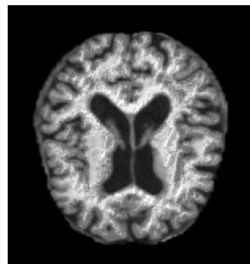


Fig.0 Attack comparizon for $\varepsilon=0.4$

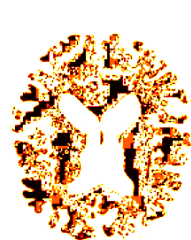
Original (Label: 1)

FGSM Perturbed $\epsilon=0.2$ 

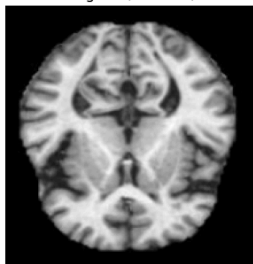
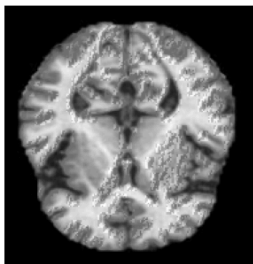
FGSM Difference

PGD Perturbed $\epsilon=0.2$ 

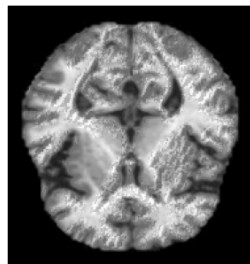
PGD Difference



Original (Label: 1)

FGSM Perturbed $\epsilon=0.2$ 

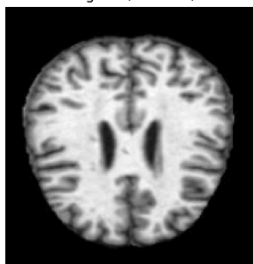
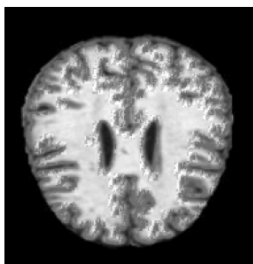
FGSM Difference

PGD Perturbed $\epsilon=0.2$ 

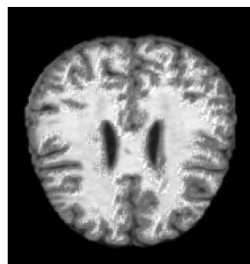
PGD Difference



Original (Label: 2)

FGSM Perturbed $\epsilon=0.2$ 

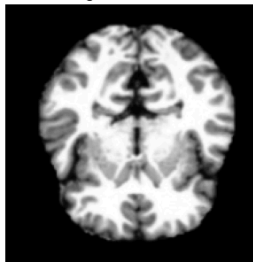
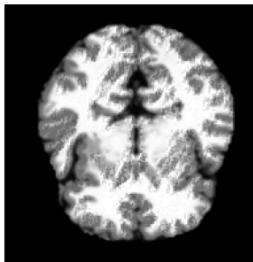
FGSM Difference

PGD Perturbed $\epsilon=0.2$ 

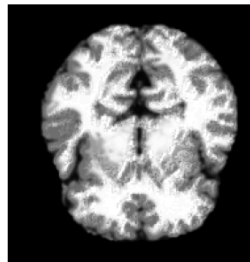
PGD Difference



Original (Label: 1)

FGSM Perturbed $\epsilon=0.2$ 

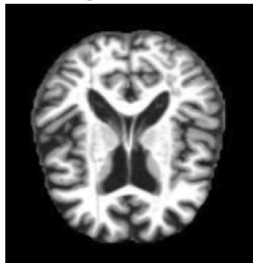
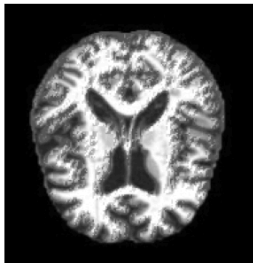
FGSM Difference

PGD Perturbed $\epsilon=0.2$ 

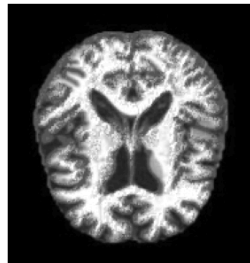
PGD Difference



Original (Label: 2)

FGSM Perturbed $\epsilon=0.2$ 

FGSM Difference

PGD Perturbed $\epsilon=0.2$ 

PGD Difference



Fig.1 Attack comparison for $\epsilon=0.2$

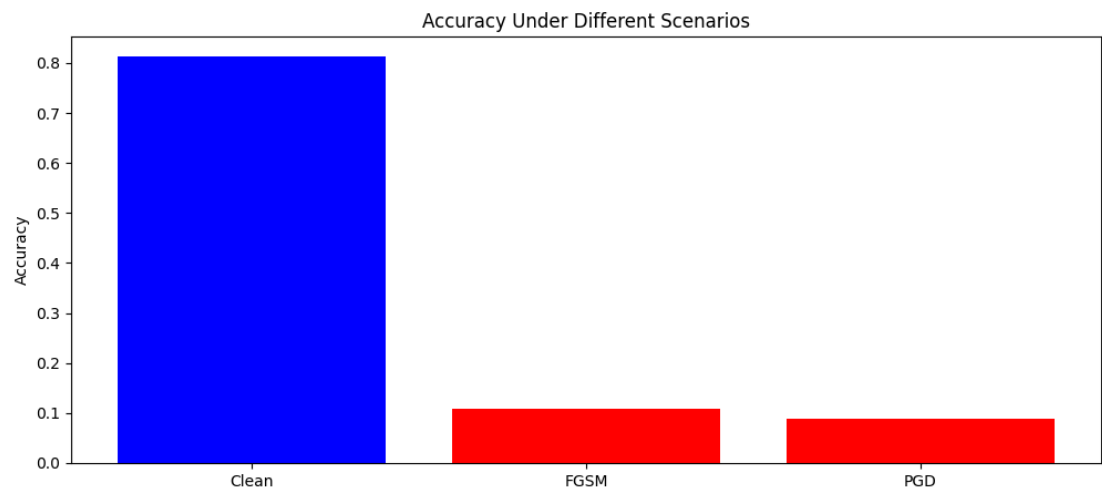


Fig.3 Accuracy comparison

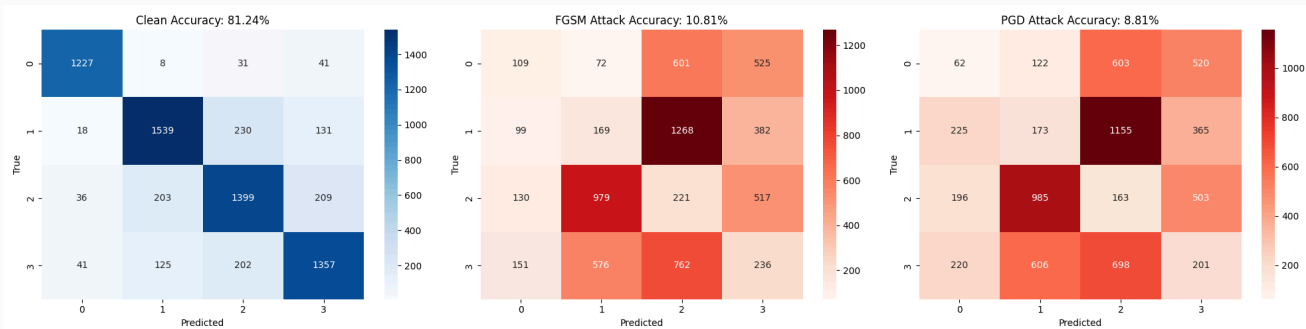


Fig.4 Confusion Matrix comparison

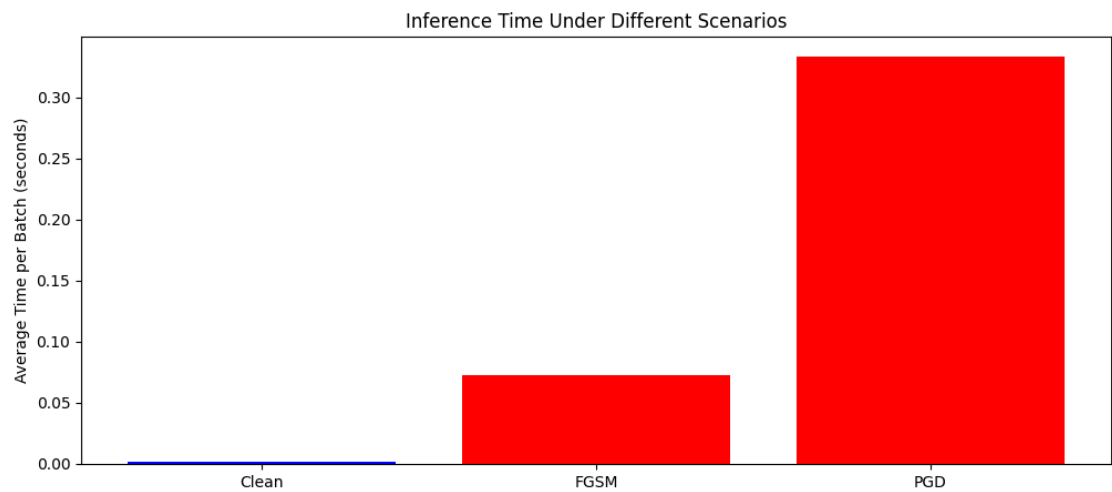


Fig.5 Inference time comparison

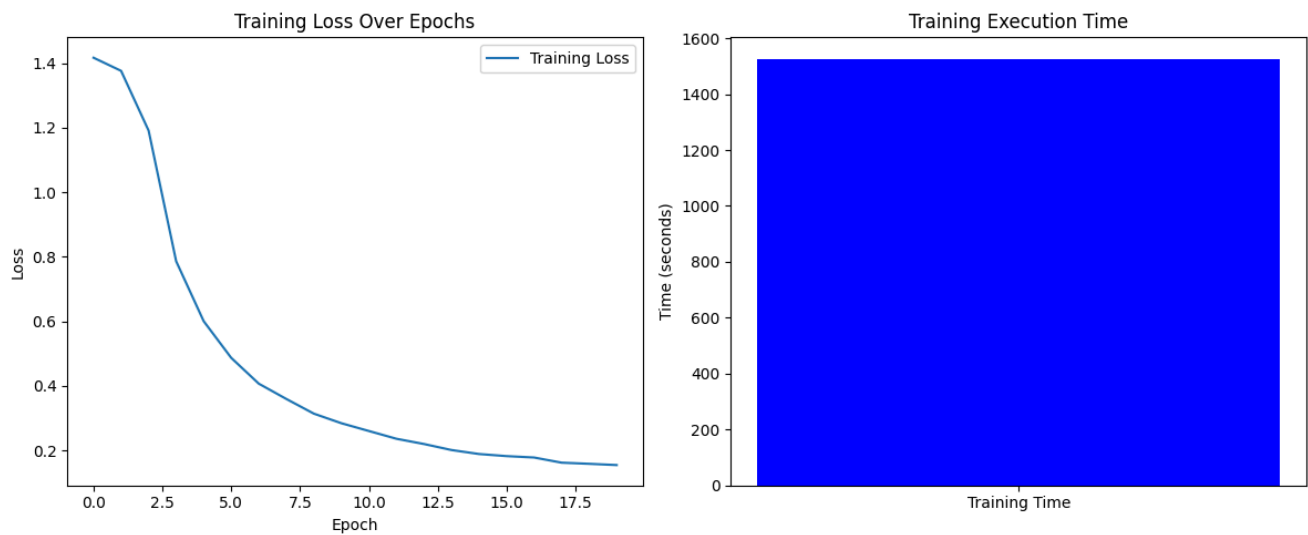


Fig.6 Training Metrics (20 epochs)

Analysis and Interpretation

As we can see, the level of perturbation ϵ plays a big part in visibility for human eyes. It's also easy to see that PGD is much more precise on contours and patterns.

Initial Model Performance

Clean Data

- Accuracy: 81.24%
- Very strong performance on class 0 (f1-score = 0.93)
- Consistent results across other classes (f1-scores between 0.75 and 0.81)
- Macro average f1-score: 0.82 → well-balanced model

Impact of Adversarial Attacks

FGSM ($\epsilon = 0.2$)

- Accuracy: 10.81% → **drop of 70.43 points**
- Severe performance degradation across all classes (f1-scores around 0.1)
- The model is easily fooled, predictions become nearly random
- Attack + inference time: 0.0724s per batch

PGD ($\epsilon = 0.2$, 10 iterations)

- Accuracy: 8.81% → **drop of 72.43 points**
- Even more destructive than FGSM
- The model completely loses its generalization ability
- Attack + inference time: 0.3330s per batch (about $\times 200$ slower than clean inference)

Conclusion

- The model performs excellently on clean data
- But it is highly vulnerable to adversarial attacks, even basic ones like FGSM
- PGD shows that minimal perturbations can entirely derail the model
- This analysis highlights the importance of considering security in deep learning systems

Difficulties Encountered

- **Problem 1:** Find another way of implementing C&W, my current algo is far too long so I don't have any results yet.

- **Problem 2:** Dataset change

Next Steps

- **Objective 1:** Implementation of C&W, DeepFool
- **Objective 2:** Maybe try with another CNN structure to see reaction
- **Objective 3:** Try these attacks on another type of data
- **Objective 4:** Go back on backdoor and more generally data poisoning attacks

FROEHLI Jean-Baptiste, Friday 25/04/2025